MODULE 07

Cart, Wishlist & Shopper Feedback

Employee Assignment & 16-Sprint Execution Plan

Project: AI-Based Product Recommendation System

Module owner: Employee 7 / Module Owner

Project duration: 8 months | **Sprint duration:** 2 weeks | **Total sprints:** 16

1. Module Objective

Provide secure cart and wishlist state plus explicit shopper feedback that can inform future recommendations. Payment processing is excluded from MVP.

2. Scope of Work

• Guest/authenticated cart operations

• Cart items and quantity updates

• Wishlist add/remove/list

• Save-for-later and like/dislike/not-interested feedback

• Ownership checks and duplicate line handling

• Link actions to recommendation impressions/events

3. Technical Responsibilities

Key data entities/tables: carts, cart_items, wishlists, wishlist_items; feedback events linked to events/impressions

API / service contract: GET/POST/PATCH/DELETE /api/v1/me/cart; GET/POST/DELETE /api/v1/me/wishlist; POST /api/v1/me/recommendation-feedback

Dependencies: Identity/session ownership contract; catalogue availability; event ingestion; recommendation impression/request IDs.

4. Sprint-Wise Employee Task Plan

Each sprint is two weeks. The owner should update task status daily, raise blockers early, submit code for review, and demonstrate completed work at sprint review. Dates are to be assigned by the project manager.

Sprint	Sprint focus	Employee tasks / implementation	Sprint deliverables
Sprint 1	Shopper state requirements	Define cart/wishlist journeys, guest behaviour, merge rules, quantity limits and feedback options; explicitly exclude payment processing.	User stories, workflow diagrams and scope boundary.
Sprint 2	Schema and contracts	Design carts/items, wishlists/items and feedback payloads; define unique constraints and ownership rules.	ERD, API contract and validation rules.
Sprint 3	Cart foundation	Implement cart creation/retrieval, add item, quantity update and remove item operations.	Cart APIs and unit tests.
Sprint 4	Ownership and guest sessions	Enforce user/session ownership; define safe guest-to-user cart merge and conflict rules.	Ownership tests and guest cart flow.

Sprint	Sprint focus	Employee tasks / implementation	Sprint deliverables
Sprint 5	Wishlist foundation	Implement wishlist add/remove/list with duplicate handling and catalogue validation.	Wishlist APIs and tests.
Sprint 6	Feedback capture	Implement like/dislike/not-interested/save-for-later feedback and link it to product/request/impression where available.	Feedback API and event mapping.
Sprint 7	Event integration	Publish cart/wishlist/feedback actions to event ingestion with idempotency and consent rules.	Event integration tests.
Sprint 8	Recommendation feedback rules	Document and implement how explicit feedback changes preference features or excludes items; avoid immediate unsupported assumptions.	Feedback policy and recommender integration.
Sprint 9	Frontend cart and wishlist	Build cart/wishlist screens, quantity controls, empty states and error handling.	React cart/wishlist flows.
Sprint 10	Feedback UI	Add feedback controls to recommendation cards and product surfaces; ensure accessible labels and undo behaviour if scoped.	Feedback UI and component tests.
Sprint 11	Cross-module integration	Verify product status changes, unavailable items, session expiry and recommendation impression linkage.	Integration test report.
Sprint 12	Concurrency and edge cases	Test duplicate requests, concurrent updates, invalid quantities, deleted products and stale sessions.	Edge-case suite and defect fixes.
Sprint 13	Security and privacy	Test cross-user access, IDOR, guest session abuse, feedback export permissions and minimal event payloads.	Security test report.
Sprint 14	Staging release	Deploy API/UI, run migrations and verify sample shopper journeys in staging.	Staging sign-off checklist.
Sprint 15	UAT and KT	Validate cart/wishlist/feedback flows; document business rules, APIs, failure cases and no-payment scope.	UAT results and KT page.
Sprint 16	Final handover	Demonstrate guest and signed-in flows, ownership protection and feedback-to-event path.	Release-ready module and handover.

5. Module-Level Acceptance Criteria

• Users cannot modify another user's cart or wishlist.

• Duplicate cart lines are handled consistently.

• Feedback is linked to recommendation context where available.

• No payment processing is included in MVP.

6. Required Testing

Unit tests: validators, business rules, transformations and core service logic. Integration tests: database migrations, API contracts, cross-module dependencies and error handling. End-to-end tests: the module's main shopper/admin/analyst journey. Negative tests: invalid inputs, unauthorized access, missing dependencies and failure/fallback paths. Performance/security tests should be added where relevant.

7. Shared Engineering Standards (Required in Every Sprint)

• Use the agreed Git branching and pull-request process; obtain at least one reviewer before merge.

• Keep configuration in environment variables; commit only a safe .env.example, never credentials or private datasets.

• Add or update automated tests with each feature; do not leave failing tests unresolved at sprint close.

• Update API contracts, schema/data dictionary, README and module notes whenever behaviour changes.

• Log useful request/job identifiers while masking personal identifiers and secrets.

• Participate in shared QA, security, Docker, CI/CD, staging, monitoring and release checks.

8. Sprint Ceremonies & Reporting

Planning: confirm sprint goal, story owners, dependencies and acceptance criteria. **Daily:** update task board and raise blockers. **Review:** demo working software, show tests and list incomplete work. **Retrospective:** record one improvement and assign an owner. **Weekly manager update:** completed tasks, in-progress tasks, blockers, risks and next steps.

9. Definition of Done

• Code is reviewed and merged to the agreed branch.

• Unit/integration tests pass and test evidence is attached to the sprint ticket.

• API/schema documentation and sample payloads are updated.

• Security and privacy implications are reviewed.

• Feature is demonstrated in a shared development or staging environment.

• Known limitations, defects and follow-up tasks are recorded.

10. Employee KT & Handover Checklist

• Explain module purpose, boundaries, architecture and dependencies.

• Walk through repository folders, entry points, data flow and key functions.

• Demonstrate one successful flow and one failure/fallback scenario.

• Run tests live and show logs/monitoring relevant to the module.

• Share setup steps, environment variable names (not secret values), migrations and commands.

• Provide API contracts, data dictionary, known issues, deployment notes and rollback/fallback steps.

• Assign a backup owner and complete handover Q&A.

Employee acknowledgement

Employee name: _____ Signature: _____ Date: _____

Manager / reviewer: _____ Review date: _____